



AULA 3

Prática de código de análise exploratória de dados em Python



Objetivos de aprendizagem

Ao final desta aula, você irá:

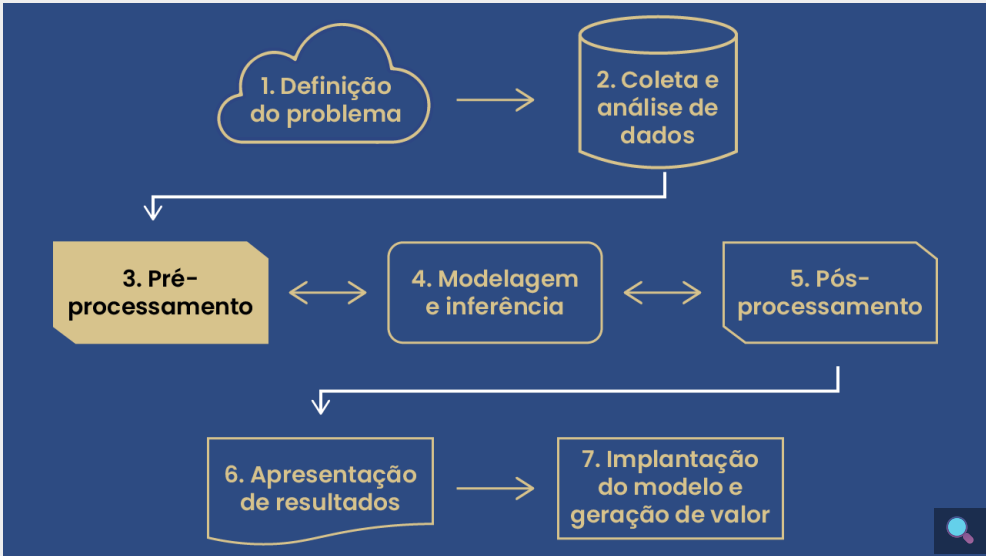
- Compreender o que é a etapa de pré-processamento de dados no contexto de projetos de ciência de dados e entender sua importância.
- Entender os conceitos de preparação, limpeza e tratamento de dados.
- Conhecer as principais técnicas de pré-processamento de dados e entender quando utilizar cada uma delas.
- Entender na prática como realizar operações de pré-processamento de dados.

Nesta terceira aula, focaremos na etapa de pré-processamento de dados, compreendendo sua importância no contexto de projetos de ciência de dados. Também exploraremos alguns conceitos de aquisição, preparação, limpeza e tratamento de dados, conheceremos algumas técnicas de pré-processamento de dados e quando cada uma deve ser utilizada. Em seguida, iremos para a parte prática da aula, acompanhando exemplos práticos de pré-processamento de dados.

Que história é essa?

Pré-processamento de dados

Lembrando do esquema básico de um projeto de ciência de dados, apresentando na aula 1, nesta terceira aula iremos focar na etapa de **pré-processamento de dados**, como ilustra a figura a seguir:



Após coletar e analisar os dados na etapa anterior, é necessário **limpar, transformar e apresentar melhor os seus dados**, a fim de obter, na próxima etapa, os melhores resultados possíveis nos algoritmos de *machine learning* ou simplesmente apresentar dados mais confiáveis para os clientes em soluções de *business intelligence*.

Após examinar com cuidado os seus dados, você pode identificar alguns problemas que devem ser tratados na etapa de pré-processamento, clique nos botões para saber mais:



Interativo

Infográfico com quatro caixas coloridas: uma azul, com o texto unidades de medida e a representação gráfica de um monitor apresentando um calendário em sua tela; uma vermelha com o texto valores faltantes e a representação gráfica de um monitor apresentando um ponto de interrogação em sua tela; uma caixa amarela com o texto valores inconsistentes e a representação gráfica de um monitor apresentando um ponto de exclamação em sua tela; e uma caixa preta com o texto intervalo dos valores e a representação gráfica de um monitor apresentando uma seta apontando para duas direções em sua tela. Ao clicar nas caixas, vemos o seguinte texto: Unidades de medida: Deve-se observar a unidade das variáveis (idade em meses ou anos, renda em dólares ou reais, por exemplo) para fazer as devidas conversões, se necessário. Valores faltantes: Caso um atributo tenha muitos registros faltantes, ele não deve ser utilizado como entrada de um modelo sem um tratamento apropriado. Valores inconsistentes: É importante entender se os valores inconsistentes são valores inválidos ou outliers. Valores inválidos: ocorrem quando temos um valor negativo em um campo que só pode ser positivo ou um texto quando se espera um número. Outliers: são valores fora do intervalo esperado. Intervalos dos valores: Deve-se prestar atenção no intervalo dos valores das variáveis. Um grande intervalo dos valores pode ser um problema para alguns modelos, podendo ser necessário transformar essa coluna usando uma transformação logarítmica para reduzir o seu intervalo de valores. Já variáveis com todos os valores em intervalos pequenos provavelmente não serão uma informação útil para os modelos. Vale lembrar que o intervalo de valores adequado varia de acordo com o domínio da aplicação.

Para ilustrar a importância da etapa de pré-processamento de dados, vamos examinar um *dataset* simplificado do **banco de crédito alemão**, adaptado do *UCI Machine Learning Repository*.

Aprenda mais!

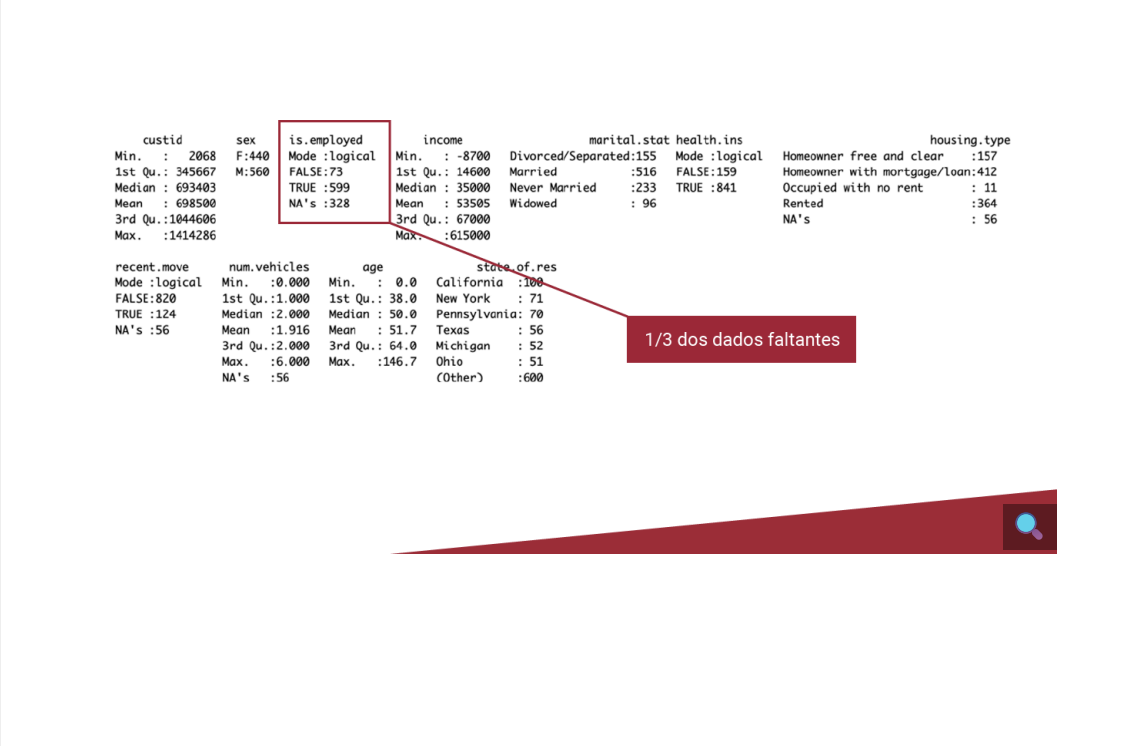
Este *dataset* é adaptado do *UCI Machine Learning Repository*, que você pode acessar [clikando aqui](#).

A figura a seguir ilustra um trecho desses dados:

	custid	sex	is.employed	income	marital.stat	health.ins	housing.type	recent.move	num.vehicles	age	state.of.res
1	2068	F	NA	11300	Married	TRUE	Homeowner free and clear	FALSE		2	49 Michigan
2	2073	F	NA		0 Married	TRUE	Rented	TRUE		3	40 Florida
3	2848	M	TRUE	4500	Never Married	FALSE	Rented	TRUE		3	22 Georgia
4	5641	M	TRUE	20000	Never Married	FALSE	Occupied with no rente	FALSE		0	22 New Mexico
5	6369	F	TRUE	12000	Never Married	TRUE	Rented	TRUE		1	31 Florida
6	8322	F	TRUE	180000	Never Married	TRUE	Homeowner with mortgage/loan	FALSE		1	40 New York
7	8521	M	TRUE	120000	Never Married	TRUE	Homeowner free and clear	TRUE		1	39 Idaho

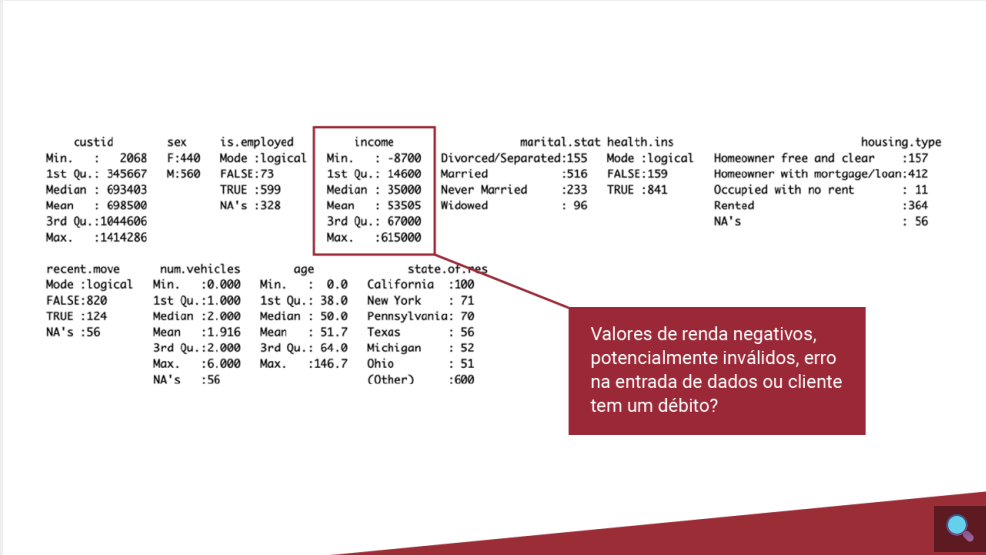
Vamos examinar os dados para verificar a necessidade de operações de pré-processamento?

Observando a área destacada na figura a seguir, nota-se que um terço dos dados da coluna **is.employed** (que mapeia se a pessoa está ou não empregada) está categorizado como **NA's (not available)**, ou seja, com valores faltantes:



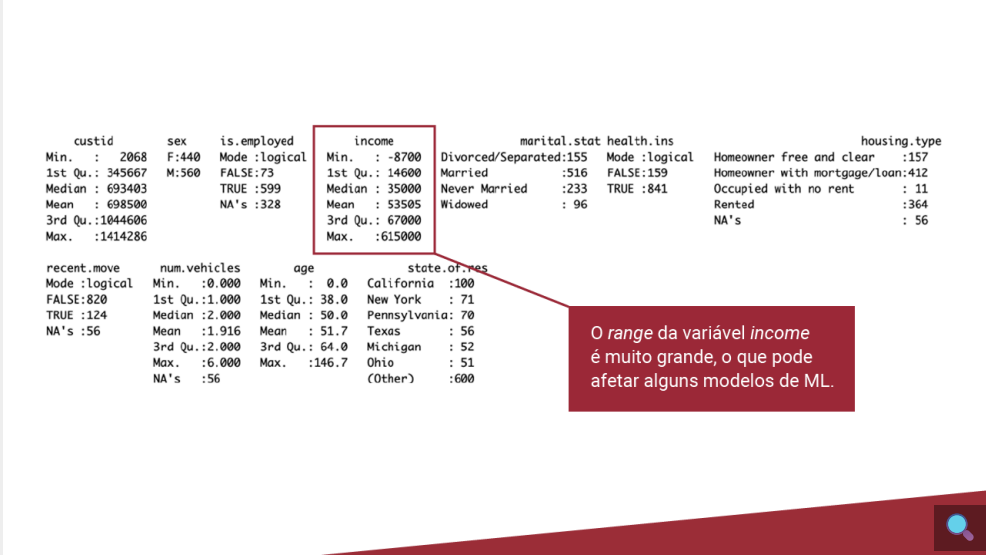
São possíveis opções de tratamento para essa coluna **realizar a sua completa exclusão do dataset** ou **substituir os valores FALSE, TRUE e NA**, respectivamente, por **NÃO, SIM** e **DESCONHECIDO**.

Agora, vamos examinar a coluna *income* (renda). Você pode notar que há **valores de renda negativos** (o mínimo é -8.700), que são potencialmente inválidos. Terá ocorrido um erro na entrada desses dados, ou a renda negativa significa que o cliente tem débitos, como empréstimos ou financiamentos?



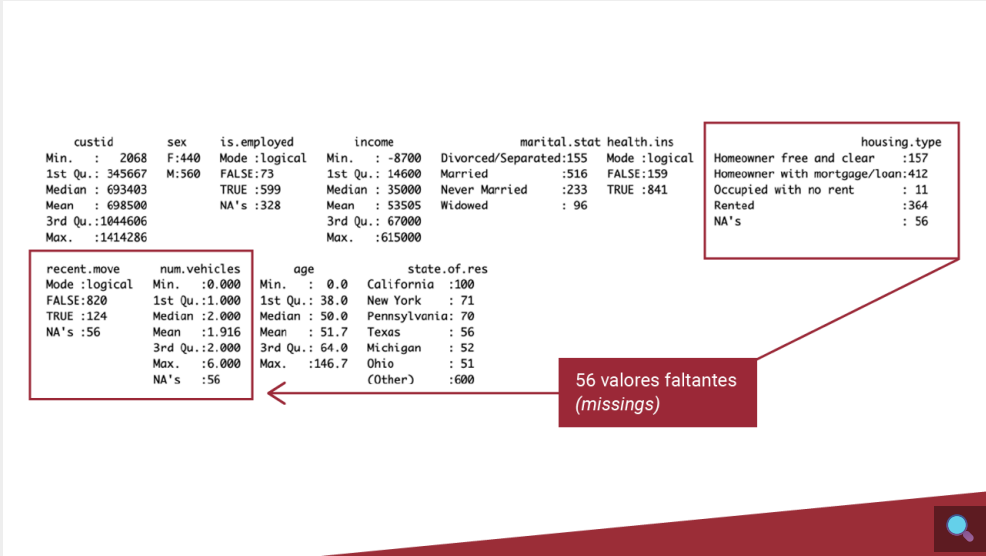
São possíveis opções para o tratamento dessa coluna simplesmente **excluir do dataset as linhas com income negativo** ou **substituir esses valores por zero**.

Outro potencial problema da coluna *income* é que **seu intervalo de valores (range) é grande demais**. Isso pode afetar negativamente alguns modelos de *Machine Learning*:



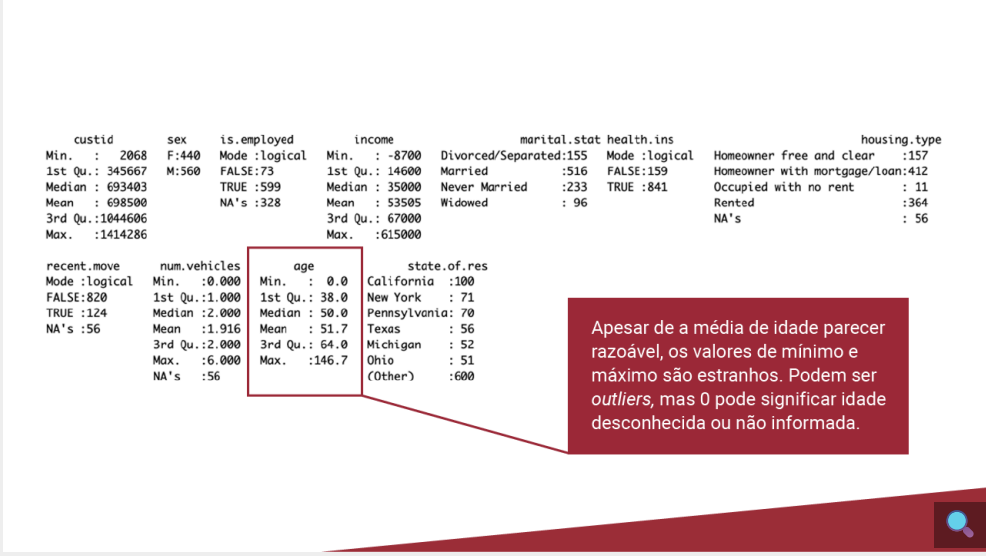
Dessa forma, pode ser uma boa ideia aplicar uma **transformação logarítmica** nesses dados para reduzir o *range* de valores.

Analisando as colunas *housing.type*, *recent.move* e *num.vehicles*, podemos notar 56 linhas com valores faltantes (*missings*), representados pelo valor *NA*:



Como são poucas linhas em relação ao tamanho total do *dataset* (5,6% do total), podemos simplesmente **excluí-las** ou, então, **substituir os valores faltantes pela média (ou mediana)** dos demais valores da coluna.

Por fim, ao analisar a coluna *age*, os valores de mínimo (0.0) e máximo (146.7) chamam a atenção.



É possível que o banco tenha clientes muito jovens ou muito idosos, mas poderíamos **normalizar as idades pela média** (se a idade absoluta não for importante para esse problema) ou, então, **converter essa variável numérica em categorias (*bins*)**.

Em seguida, vamos conhecer em mais detalhes quais são as técnicas utilizadas no pré-processamento de dados.



Técnicas de pré-processamento de dados

Existem diversas técnicas de pré-processamento que podem ser utilizadas para preparar um *dataset* para a criação de modelos de *Machine Learning* ou mesmo análises de *Business Intelligence*.

Mantendo o foco

É importante ressaltar que, antes de aplicar as operações de pré-processamento nos dados, recomenda-se fazer uma cópia dos dados fontes originais e, após a aplicação das operações de pré-processamento, **organizar os dados em um novo arquivo único tratado**.

Algumas das principais técnicas serão detalhadas a seguir.

Limpeza

De forma resumida, a **limpeza** consiste na verificação da consistência das informações, correção de possíveis erros de preenchimento ou eliminação de valores desconhecidos, redundantes ou não pertencentes ao domínio.



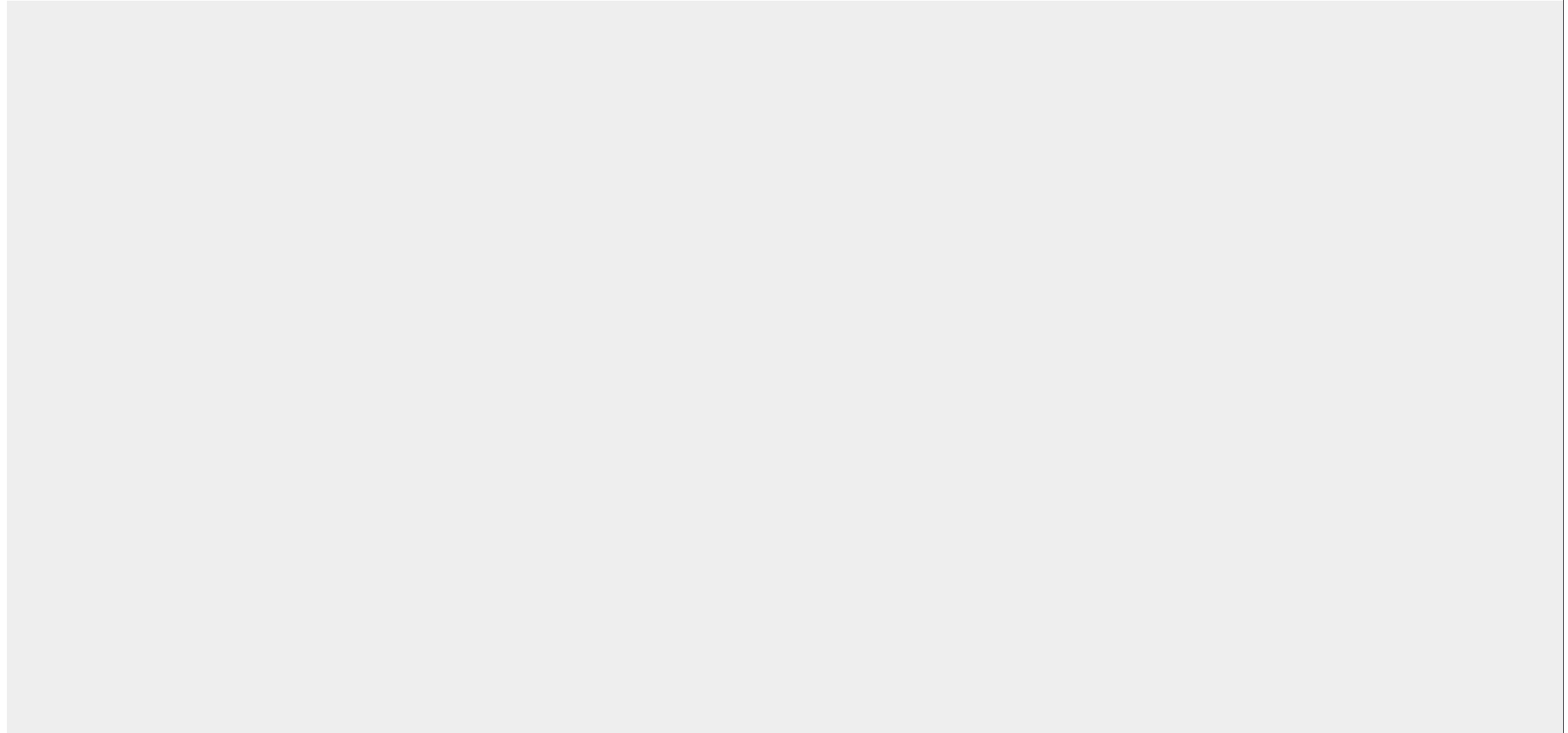
Quando trabalhamos com dados reais, é muito comum que eles estejam **incompletos, ruidosos ou inconsistentes**. A limpeza é muito importante, pois, quanto pior a qualidade dos dados, pior será a qualidade dos modelos gerados (*garbage in, garbage out*).

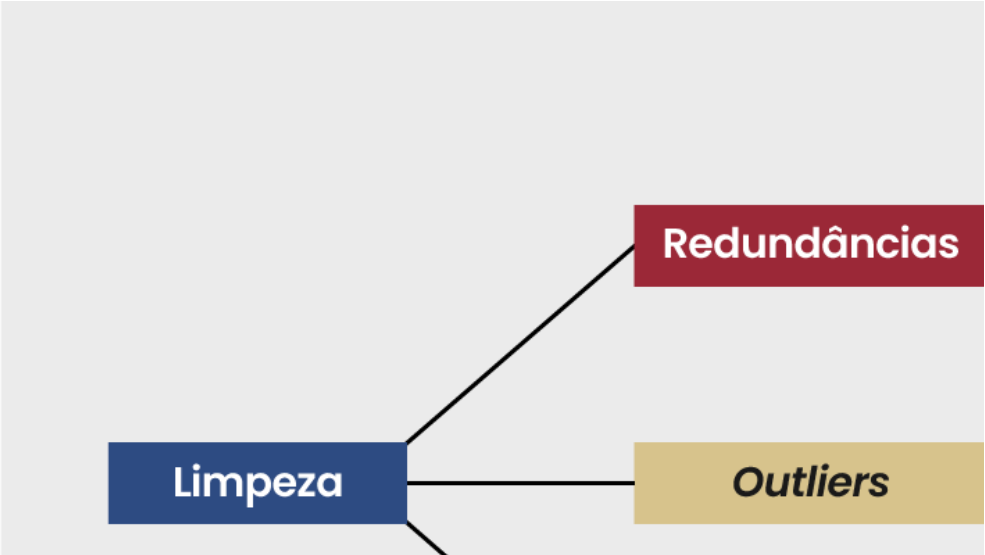
Fique ligado!

Vale lembrar que a melhor maneira de evitar a poluição de dados é organizando sua entrada, mas isso nem sempre é possível, reforçando a importância da **realização da limpeza dos dados** trabalhados.

Para que a limpeza de dados seja feita de forma adequada, é essencial o envolvimento dos especialistas de negócio para apoiar na limpeza de informações ausentes (exclusão de casos, preenchimento manual, preenchimento com valores globais constantes, preenchimento com medidas estatísticas, preenchimento por meio de métodos de *Machine Learning*) ou mesmo na limpeza de inconsistências e valores não pertencentes ao domínio (exclusão de casos, correção de erros, etc.).

A figura a seguir ilustra os principais casos que podem ser tratados com a técnica de limpeza. Clique nos locais indicados para acompanhar a explicação:





Interativo

Neste interativo, vê-se uma caixa azul, escrita Limpeza, conectada a três outras caixas: uma vermelha, escrita Redundâncias ; uma amarela, escrita Outliers, e uma preta, escrita Missings. Ao clicar nas caixas, lê-se o seguinte texto: Redundâncias. Podemos ter redundâncias que precisam ser eliminadas, seja de linhas ou colunas (características). Outliers. Também podemos ter possíveis outliers, podendo ser indicados por valores extremos. Missings são os valores faltantes, e será necessário identificá-los (marcá-los) e, em seguida, tratá-los, por exemplo, com um valor adequado.

Agregação

A **agregação** consiste em combinar dois ou mais objetos em um único.

Um exemplo disso seria agregar os registros de transações de vendas diárias de produtos em lojas distintas em uma única transação mensal por loja, como ilustra a figura a seguir:

Item	Local	Data	Preço
Relógio	Paris	09/06/2004	100
Pilha	Rio	09/06/2004	3
Vestido	Rio	09/06/2004	50
Sapatos	Paris	09/06/2004	35

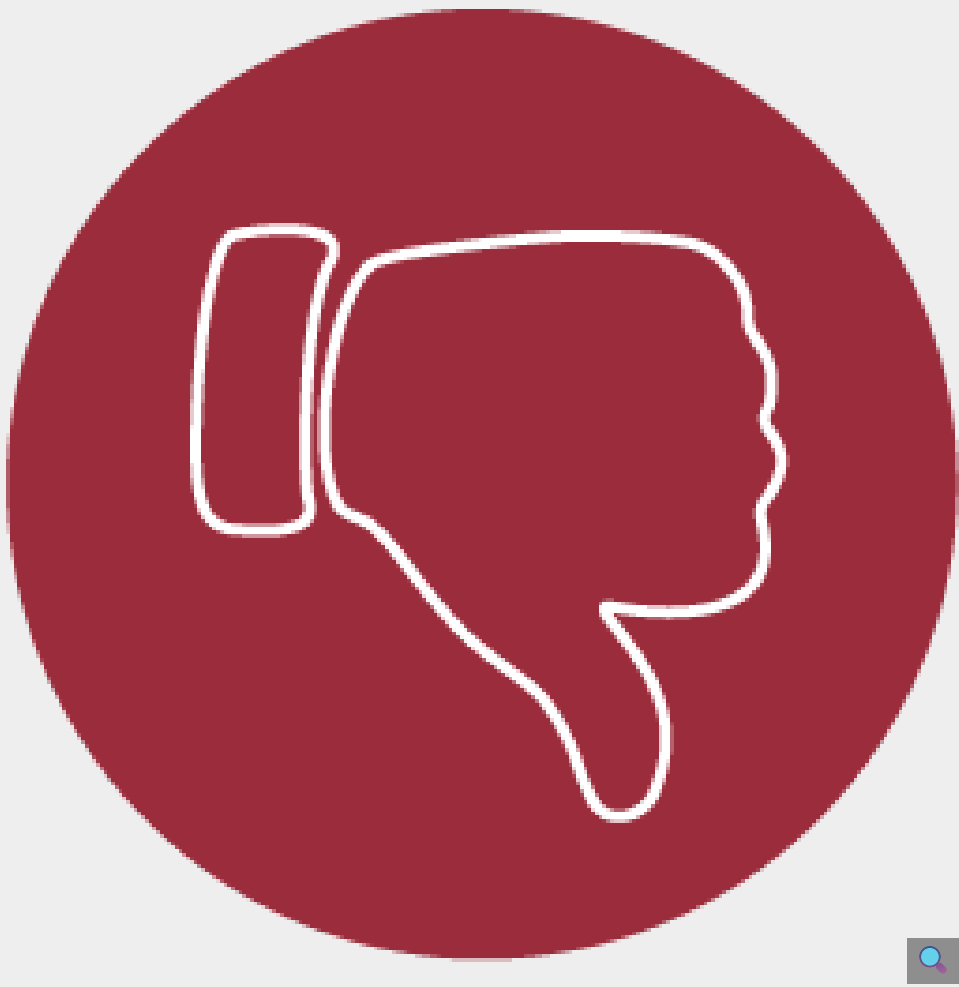
↓

Local	Data	Valor
Paris	jun/04	135
Rio	jun/04	53



Vantagens

- Um conjunto de dados menor tem menor custo de processamento.
- A agregação pode atuar como uma mudança de granularidade (muito usada em OLAP).



Desvantagens

- Podemos perder detalhes que podem ser interessantes (por exemplo, identificar o dia da semana que tem mais vendas).

Amostragem

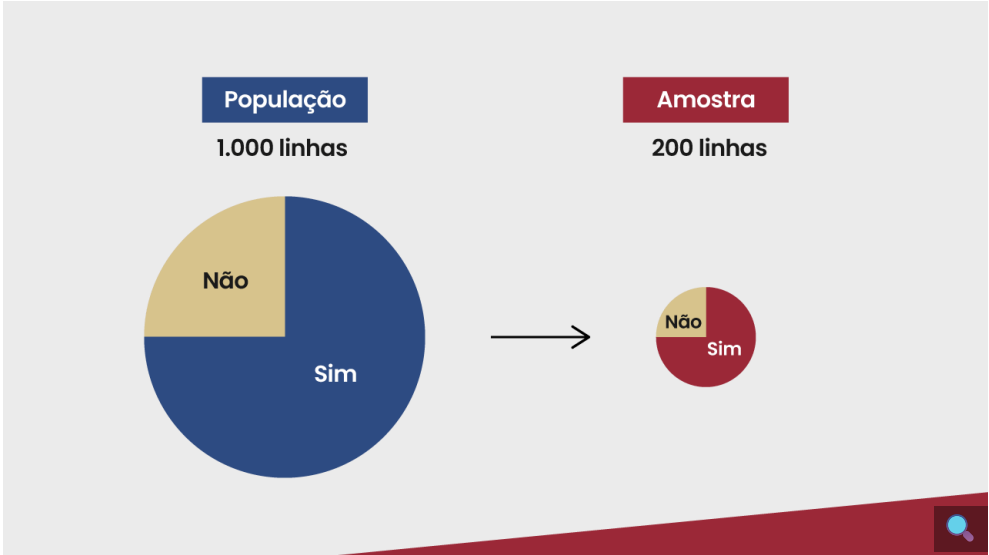
A **amostragem** consiste em selecionar um subconjunto dos dados a serem analisados ou trabalhados.



O princípio chave da amostragem é a ideia de que usar uma amostra funcionará quase tão bem quanto usar o conjunto inteiro de dados, **desde que a amostra seja representativa**, ou seja, se tiver aproximadamente a mesma propriedade de interesse do conjunto original de dados (por exemplo, a média).

São possíveis abordagens de amostragem a **amostragem aleatória simples** (que pode ser com ou sem substituição) e a **amostragem estratificada** (que acomoda frequências diferentes).

A amostragem estratificada é ilustrada na figura a seguir:



Em *Machine Learning*, um exemplo da utilização da técnica de amostragem é quando dividimos nosso *dataset* em dois conjuntos, um de **treinamento**, para construir o modelo, geralmente maior, e um de **teste**, para avaliar o modelo construído, geralmente menor.

Mantendo o foco

Esta operação se chama **holdout** e é importante porque, após construído, um modelo precisa ser validado com dados diferentes dos utilizados na sua construção.

A figura a seguir ilustra o *holdout*:

12/02/2026, 21:20

Análise exploratória e pré-processamento de dados

Cliente	Ano	Mês	Dia	Atributos	Pagou?
João	2012	Janeiro	1	...	Sim
Oswaldo	2012	Janeiro	9	...	Sim
Maria	2012	Janeiro	21	...	Não
Roberta	2012	Fevereiro	4	...	Sim
...	...	...	...	...	...
Thiego	2013	Novembro	29	...	Sim
Dalva	2013	Dezembro	2	...	Não
Josy	2013	Dezembro	11	...	Não
Érica	2014	Janeiro	11	...	Sim
Renata	2014	Janeiro	19	...	Sim
...	...	...	...	...	...
Dorival	2014	Dezembro	20	...	Sim

Treinamento

Teste

Outro exemplo de utilização da técnica de amostragem é a validação cruzada (*Cross validation*), que pode ser estratificada ou não, e é utilizada para avaliar modelos de *Machine Learning*.

Cross validation

Acompanhe como ela funciona:

- A validação cruzada consiste em dividir aleatoriamente o conjunto de dados com n elementos em k subconjuntos disjuntos com aproximadamente o mesmo número de elementos (ou seja, cada um com aproximadamente n/k).
- A cada rodada, cada um dos k subconjuntos é usado como conjunto de teste, e os conjuntos restantes são reunidos em um único conjunto de treino.
- O processo é repetido k vezes, sendo, então, gerados e avaliados k modelos, e essas avaliações são geralmente consolidadas por meio da média. Os valores mais comuns de k são 3, 5 e 10.

A **validação cruzada estratificada** é similar, mas, ao gerar os subconjuntos, a proporção de exemplos em cada uma das classes é considerada durante a amostragem.

Vamos entender como seria a **validação cruzada 3-fold**?
Acompanhe.

Validação cruzada 3-fold

Passo 1

Embaralhar os índices, como é feito a seguir:

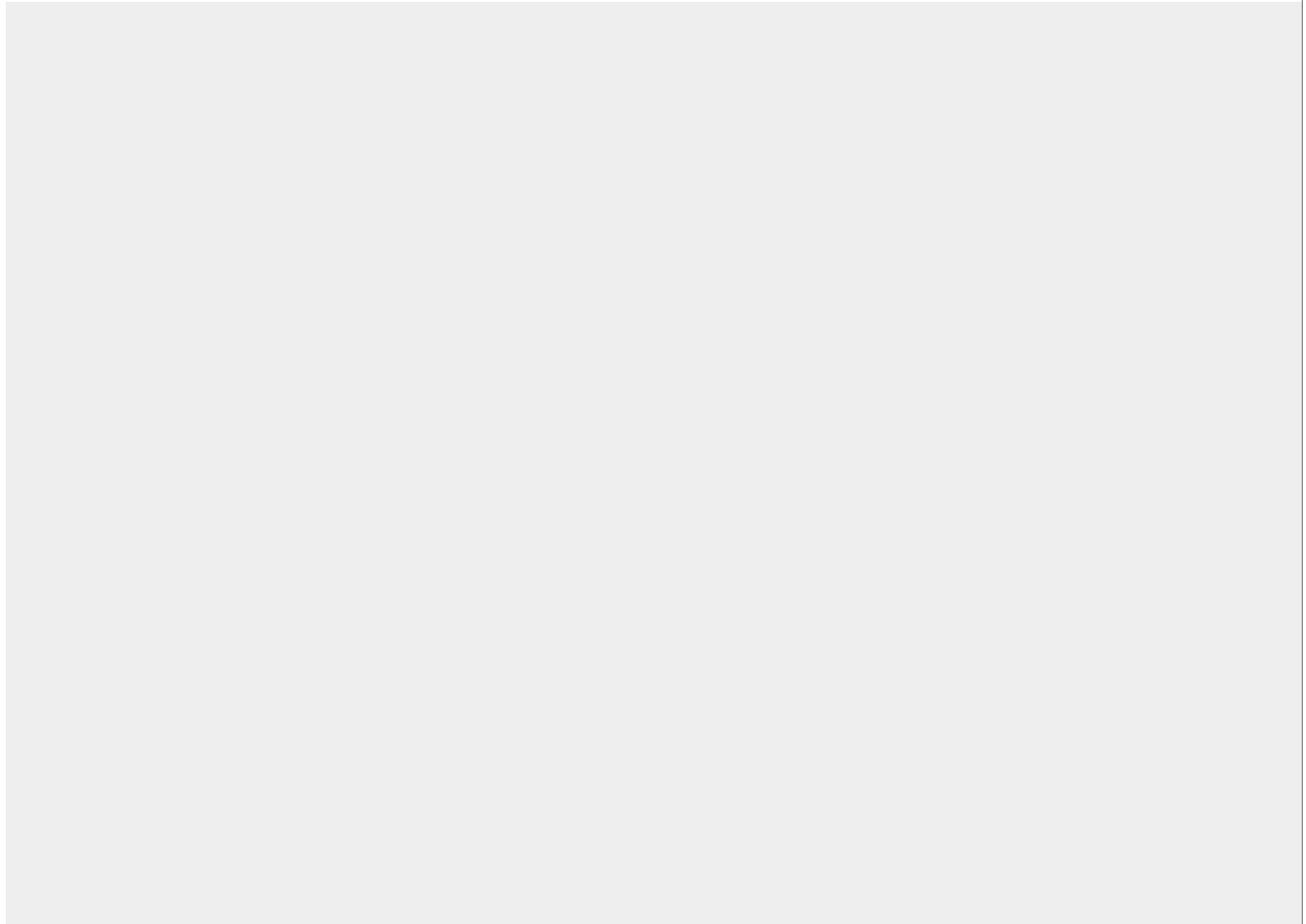
Índice	Nome	...	Índice	Nome	...
1	Adriano	...	9	Maria	...

A validação cruzada garante melhor estimativa da acurácia de teste do modelo, pois, em vez de ter a acurácia de teste medida com apenas uma parte do *dataset* (como no *holdout*, que utiliza apenas o conjunto de teste para tal), o modelo é testado, de alguma forma, **com todos os dados do *dataset***, o que resulta em melhor aproveitamento do *dataset*.

Mantendo o foco

Vale a pena ressaltar que, principalmente quando temos um *dataset* desbalanceado (por exemplo, com 90% dos dados pertencendo à classe 1 e 10% pertencendo à classe 2), é essencial utilizar a validação cruzada estratificada.

Para ilustrar esse ponto, clique nas setas e observe o *dataset* desbalanceado da figura a seguir:



Dataset desbalanceado

Índice	Nome	Peso (kg)	Altura (cm)
1	Adriano	80	185
2	Mateus	67	175
3	Thiago	58	187
4	João	82	176
5	Pedro	74	168

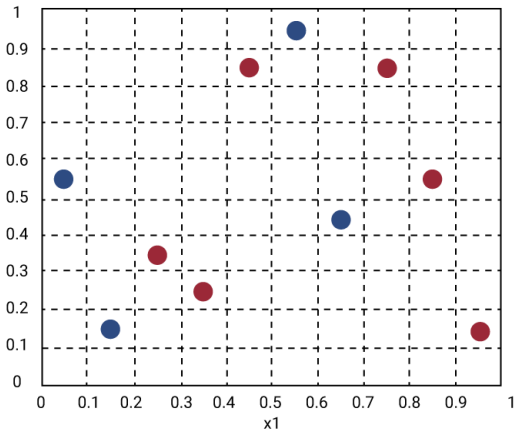
Interativo

Interativo onde, no primeiro slide, vê-se uma tabela intitulada Dataset desbalanceado. A tabela contém as seguintes colunas: Índice: uma numeração sequencial de 1 a 12. Nome: com nomes de indivíduos, os mesmos vistos na imagem anterior, como Adriano, Mateus, Thiago, João, Pedro, Lucas, Paulo, Timóteo, Maria, Martha, Madalena e Esther. Peso (kg): listando o peso de cada indivíduo em quilogramas. Altura (cm): a altura de cada indivíduo em centímetros. Sexo: indicando o gênero de cada indivíduo, com 'M' para masculino e 'F' para feminino. Também há uma nota no lado direito que informa: 8 homens igual a 67 por cento da base de dados, 4 mulheres igual a 33 por cento da base de dados. No segundo slide, lê-se: nesse caso, para executar a validação cruzada estratificada, o primeiro passo é embaralhar os índices de forma estratificada. Abaixo, vê-se os mesmos índices e nomes do slide anterior; à esquerda, eles estão em ordem crescente, de acordo com o índice: 1 Adriano, 2 Mateus, 3 Thiago, 4 João, 5 Pedro, 6 Lucas, 7 Paulo, 8 Timóteo, 9 Maria, 10 Martha, 11 Madalena e 12 Esther. À direita, conectado por uma seta azul, vê-se a mesma tabela, dessa vez ordenada aleatoriamente, com os índices seguindo a ordem: 5, 4, 8, 3, 6, 7, 1, 2, 11, 9, 12 e 10. No slide 3, lê-se o texto: em seguida, formamos os três conjuntos. Repare que a proporção de homens e mulheres é proporcional nos conjuntos de treino e teste. Abaixo, vemos a mesma tabela do slide anterior, ordenada de forma aleatória. Além dos índices e nomes, vemos a coluna Pasta 1, indicada também à esquerda por um círculo vermelho com o número 1 no centro, que divide as pessoas entre Treino e Teste. No treino, vemos: 5 Pedro, 4 João, 8 Timóteo, 3 Thiago, 6 Lucas, 11 Madalena, 9 Maria e 12 Ester. No Teste, vemos 7 Paulo, 1 Adriano, 2 Mateus e 10 Martha. No slide 4, vemos a pasta 2, indicada também à esquerda por um círculo vermelho com o número 2 no centro, que apresenta no treino: 5 Pedro, 4 João, 7 Paulo, 1 Adriano, 2 Mateus, 11 Madalena, 9 Maria e 10 Martha e no teste: 8 Timóteo, 3 Thiago, 6 Lucas e 12 Ester. No slide 5, vemos a pasta 3, indicada também à esquerda por um círculo vermelho com o número 3 no centro, que apresenta no treino: 8 Timóteo, 3 Thiago, 6 Lucas, 7 Paulo, 1 Adriano, 2 Mateus, 12 Ester e 10 Martha; e no teste: 5 Pedro, 4 João, 11 Madalena e 9 Maria.

Redução da dimensionalidade

O número de características de um *dataset* pode ser considerado a **dimensionalidade** dos dados.

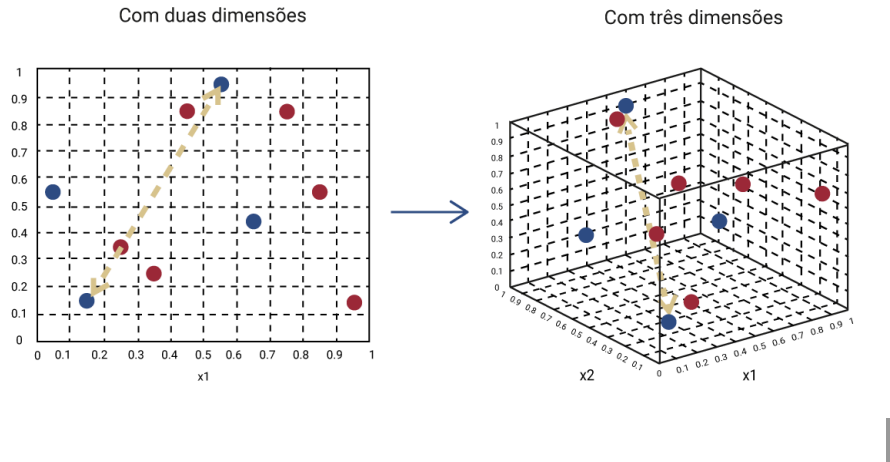
Por exemplo, duas variáveis de entrada (colunas) delimitam um espaço com duas dimensões, e cada exemplo (linha do *dataset*) é um ponto desse espaço, como ilustra a figura a seguir:



Esse raciocínio pode ser escalado para qualquer número de variáveis de entrada para criar espaços multidimensionais.

Entretanto, quanto maior o número de dimensões de um espaço, maior é o volume desse espaço, sendo provável que esse *dataset* represente uma amostragem muito esparsa e provavelmente não representativa desse espaço.

Isso é conhecido como **maldição da dimensionalidade**, ilustrada da seguinte maneira:



Para atuar nesse problema, a **redução de dimensionalidade** consiste em projetar os dados em um espaço de dimensão inferior, mas que ainda preserva as propriedades mais importantes do *dataset* original.

No mundo real, os *datasets* podem ter muitas características, e muitos algoritmos de *Machine Learning* funcionam melhor se a dimensionalidade for menor.

São exemplos da técnica de redução de dimensionalidade:

Análise de componentes principais (PCA)



Feature selection (seleção de características)



Agregação



Por fim, quais são as vantagens e as desvantagens da aplicação da técnica de redução de dimensionalidade?



Vantagens

- Um *dataset* menor tem menor custo de processamento.
- A redução de dimensionalidade pode eliminar características irrelevantes, reduzir ruídos e remover dependências lineares entre os atributos, como variáveis correlacionadas.



Desvantagens

- Caso sejam aplicadas técnicas como o PCA, as “novas” variáveis não estarão diretamente relacionadas às variáveis de entrada originais, tornando a projeção difícil de interpretar.

A seguir, conheça mais sobre a técnica de seleção de características.

Seleção de características

Também conhecida como ***feature selection***, a seleção de características consiste em usar apenas um subconjunto das características (atributos) originais, desconsiderando as características redundantes ou irrelevantes.

Para tal, existem diversas abordagens:



A figura a seguir ilustra a seleção de características:

Id	Nome	Sexo	Idade
123	Tatiana	F	16
456	Rubens	M	50
780	Adriano	M	28
990	João	M	35



Sexo	Idade
F	16
M	50
M	28
M	35

Criação de recursos

Também conhecida como ***feature engineering***, a criação de recursos consiste em criar, a partir dos atributos originais, um conjunto de atributos que capture informações importantes.

Para tal, podem ser utilizadas diversas metodologias, clique nos itens e veja:

12/02/2026, 21:20

Análise exploratória e pré-processamento de dados

Extração de características

Construção de recursos

Enriquecimento

Transformação de dados

Confira exemplos da técnica de transformação de dados a seguir.

Normalização

A normalização redimensiona os dados de um ou mais atributos do intervalo original para o intervalo de 0 a 1, com o objetivo de alterar os valores para uma escala comum, sem distorcer as diferenças nos intervalos de valores.

Mantendo o foco

Esta é uma boa técnica para usar quando não se conhece bem a distribuição dos dados ou quando se sabe que a distribuição não é gaussiana (normal).

São utilizados os valores mínimo e máximo observáveis, sendo possível estimar esses valores a partir dos dados disponíveis. Você pode normalizar os dados dividindo todos os valores pelo valor máximo encontrado ou subtraindo o valor mínimo e dividindo pelo intervalo entre os valores máximo e mínimo:

$$y = \frac{(x - \min)}{(\max - \min)}$$



A seguir, acompanhe como funciona a padronização.

Padronização

A padronização redimensiona a distribuição de um ou mais atributos dos valores observados para que eles tenham as propriedades de uma distribuição normal padrão (média 0 e desvio padrão 1).

A padronização pressupõe que suas observações sigam uma

distribuição normal. Ainda é possível padronizar seus dados mesmo se essa premissa não for verdadeira, mas a técnica é mais eficaz se os dados seguirem essa distribuição.

Para padronizar os dados, calculam-se a média estatística e o desvio padrão dos valores dos atributos, subtrai-se a média de cada valor e divide-se o resultado pelo desvio padrão.

Um valor é padronizado da seguinte maneira:

$$média = \frac{soma(x)}{contagem(x)}$$
$$desvio\ padrão = sqrt(soma(\frac{(x - média)^2}{contagem(x)}))$$
$$y = \frac{(x - média)}{desvio\ padrão}$$



A seguir, entenda mais sobre a discretização.

Discretização

A discretização consiste em transformar um atributo contínuo em categorias ordinais.

Acompanhe na figura a seguir:

Salário	Salário discretizado
R\$ 1.000,00	Baixo
R\$ 500,00	Baixo
R\$ 5.000,00	Médio
R\$ 10.000,00	Alto
R\$ 20.000,00	Alto



One-hot encoding/dummy encoding

Consiste em transformar atributos categóricos nominais (ou seja, sem ordenação existente entre eles) em “colunas binárias”. Para tal, é criada uma variável binária para cada valor único da variável original. Assim, um atributo “cor dos olhos” com os valores “azuis”, “verdes” e “castanhos” teria estes valores codificados, respectivamente, para [1,0,0], [0,1,0] e [0,0,1].

Fique ligado!

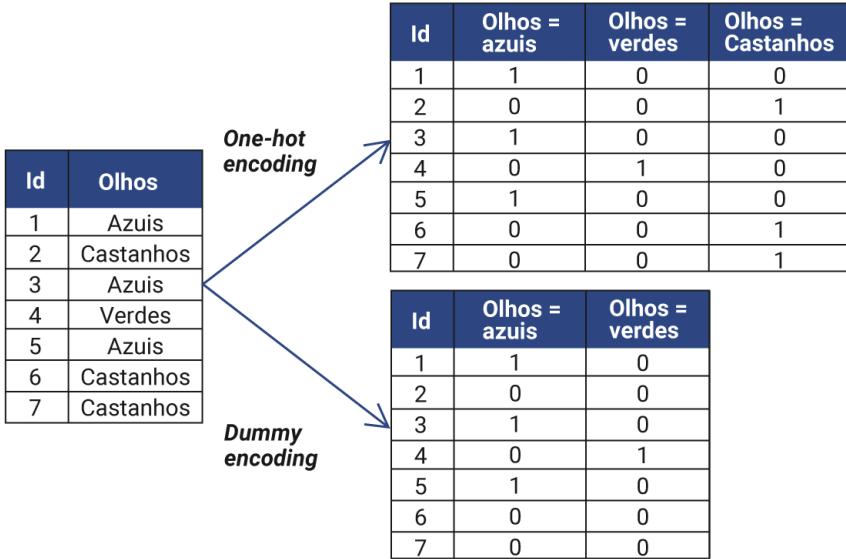
O *one-hot encoding* cria uma variável binária para cada categoria existente, mas essa representação inclui redundância.

Por exemplo:

Se soubermos que [1, 0, 0] representa 'azuis' e [0, 1, 0] representa 'verdes', não precisamos de outra variável binária [0, 0, 1] para representar 'castanhos'.

Poderíamos usar [0, 0] para 'castanhos', [1, 0] para 'azuis' e [0, 1] para 'verdes', como na codificação de variável *dummy*, que representa c categorias em $c-1$ variáveis binárias.

As duas técnicas são ilustradas pela figura a seguir:

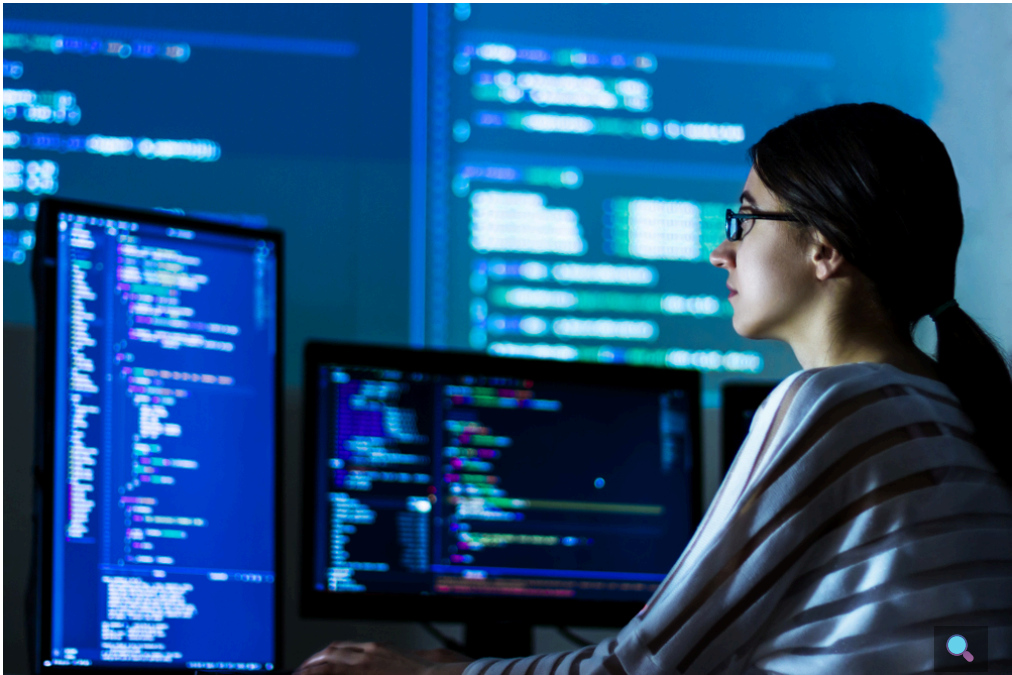


Atenção! Cuidado com o *data leakage*!

No caso da normalização e da padronização, uma abordagem ingênua (e incorreta) para preparar dados **aplica a transformação em todo o conjunto de dados** antes do particionamento em bases de treino e teste.

Fique ligado!

Isso resulta em um problema conhecido como **vazamento de dados** (*data leakage*): algum conhecimento sobre o conjunto de teste vaza para o conjunto de dados usado para treinar o modelo, podendo resultar em uma estimativa incorreta do desempenho do modelo ao fazer predições sobre novos dados.



Por exemplo, quando fazemos a normalização, convertemos as variáveis de entrada para um intervalo entre 0 e 1. Para tal, precisamos calcular os valores mínimo e máximo para cada variável e, então, reescalar as variáveis.

Porém, se fizermos primeiro isso com todo o *dataset* e depois o particionarmos em conjuntos de treino e teste, possivelmente o **conjunto de treino saberá algo sobre o conjunto de teste**, pois foram utilizados os valores mínimo e máximo globais, e não apenas os do conjunto de treino.

Para saber a forma correta de aplicar essas transformações, clique nos *cards*:

Passo 1

Saiba mais

Interativo

Interativo apresentando quatro caixas azuis, com os títulos Passo 1, 2, 3 e 4. Abaixo de cada uma delas, vê-se uma caixa amarela com o texto Saiba mais. Ao clicar nas caixas, vemos o seguinte texto: Passo 1: dividir o dataset em conjuntos de treino e teste. Passo 2: ajustar a preparação de dados no conjunto de dados de treino, por exemplo, determinar o mínimo e o máximo deste conjunto. Passo 3: aplicar a preparação de dados no conjunto de treino e teste separadamente, utilizando os valores mínimo e máximo do conjunto de treino. Passo 4: construir e avaliar modelos.

Vamos acompanhar agora um exemplo de pré-processamento de dados em Python?





Exemplo de pré-processamento de dados em Python

Para exemplificar como funciona a etapa de pré-processamento de dados, iremos utilizar novamente o *dataset* iris, que utilizamos na aula anterior, para ilustrar a limpeza de dados, em especial o tratamento de *missings*.

Observação: como as bibliotecas Python são dinâmicas e evoluem ao longo do tempo, este exemplo pode conter trechos de código desatualizados. O código fonte original, disponível no GitHub da disciplina, estará sempre atualizado.



Em seguida, iremos utilizar alguns *datasets* artificiais para falarmos sobre as operações de transformação de dados, como normalização, padronização, *one-hot encoding* e *dummy encoding*.

Iniciaremos esta prática importando os pacotes que utilizaremos:

```
cod.py

1 # Importação de pacotes
2 import pandas as pd
3 import numpy as np
4 from sklearn.preprocessing import MinMaxScaler # normalização
5 from sklearn.preprocessing import StandardScaler # padronização
6 from sklearn.preprocessing import OrdinalEncoder # ordinal encoding
7 from sklearn.preprocessing import OneHotEncoder # one-hot encoding e dummy encoding
```

Agora, iremos importar o *dataset* iris a partir de um *link* e armazenar os dados em um *dataframe*, da mesma forma que fizemos na prática da aula anterior.

Exibiremos também as cinco últimas linhas do *dataframe*:

```
cod.py

1 # importando dados uma url para um dataframe
2
3 # url a importar
4 url_dados = 'https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data'
5
6 # labels dos atributos do dataset
7 labels_atributos = ['comprimento_sepala', 'largura_sepala',
8                     'comprimento_petala', 'largura_petala', 'especie']
9
10 # carga do dataset através da url
11 iris = pd.read_csv(url_dados, names=labels_atributos)
12
13 # exibindo as últimas linhas
14 iris.tail()
```

	comprimento_sepala	largura_sepala	comprimento_petala	largura_petala	especie
145	6.7	3.0	5.2	2.3	Iris-virginica
146	6.3	2.5	5.0	1.9	Iris-virginica
147	6.5	3.0	5.2	2.0	Iris-virginica
148	6.2	3.4	5.4	2.3	Iris-virginica
149	5.9	3.0	5.1	1.8	Iris-virginica

Como vimos na aula anterior, o *dataset* iris não tem valores faltantes (*missings*). Agora, iremos criar uma cópia desse *dataframe* adicionando uma nova linha que contém um valor *missing*. Exibiremos novamente as cinco últimas linhas para verificar a nova linha que inserimos:

```
cod.py

1 # criando um novo dataframe com iris + uma nova linha com um valor missing
2 # para o próximo exemplo
3 df = iris.append({'largura_sepala': 4.0,
4                  'comprimento_petala': 5.0,
5                  'largura_petala': 0.4,
6                  'especie': 'Iris-setosa'},
7                  ignore_index=True)
8 df.tail()
```

	comprimento_sepala	largura_sepala	comprimento_petala	largura_petala	especie
146	6.3	2.5	5.0	1.9	Iris-virginica
147	6.5	3.0	5.2	2.0	Iris-virginica
148	6.2	3.4	5.4	2.3	Iris-virginica
149	5.9	3.0	5.1	1.8	Iris-virginica
150	NaN	4.0	5.0	0.4	Iris-setosa

Em seguida, iremos executar um comando para eliminar desse *dataframe* as linhas que tiverem qualquer valor *missing*. Nesse caso, será a última linha que adicionamos:

```
cod.py

1 # eliminando linhas que tenham ALGUM valor missing
2
3 df = df.dropna(how='any')
4 df.tail()
```

	comprimento_sepala	largura_sepala	comprimento_petala	largura_petala	especie
145	6.7	3.0	5.2	2.3	Iris-virginica
146	6.3	2.5	5.0	1.9	Iris-virginica
147	6.5	3.0	5.2	2.0	Iris-virginica
148	6.2	3.4	5.4	2.3	Iris-virginica
149	5.9	3.0	5.1	1.8	Iris-virginica

Agora, iremos criar novamente um *dataframe* com a cópia do dataframe original. Em seguida, adicionamos uma nova linha com um valor *missing* e uma nova linha com todos os valores *missing*:

cod.py

```
1 # criando (novamente) um novo dataframe com iris + uma nova linha com um
2 # valor missing para o próximo exemplo
3 df = iris.append({'largura_sepala': 4.0,
4                  'comprimento_petala': 5.0,
5                  'largura_petala': 0.4,
6                  'especie': 'Iris-setosa'},
7                  ignore_index=True)
8
9 # adicionando uma nova linha com todos os valores missing para o próximo exemplo
10 df = df.append({}, ignore_index=True)
11 df.tail()
```

	comprimento_sepala	largura_sepala	comprimento_petala	largura_petala	especie
147	6.5	3.0	5.2	2.0	Iris-virginica
148	6.2	3.4	5.4	2.3	Iris-virginica
149	5.9	3.0	5.1	1.8	Iris-virginica
150	NaN	4.0	5.0	0.4	Iris-setosa
151	NaN	NaN	NaN	NaN	NaN

Agora, iremos eliminar desse *dataframe* as linhas que tiverem todos os valores *missing*. Nesse caso, será removida a última linha que adicionamos, mas não a anterior:

cod.py

```
1 # eliminando linhas que tenham TODOS os valores missing
2 df = df.dropna(how='all')
3 df.tail()
```

	comprimento_sepala	largura_sepala	comprimento_petala	largura_petala	especie
146	6.3	2.5	5.0	1.9	Iris-virginica
147	6.5	3.0	5.2	2.0	Iris-virginica
148	6.2	3.4	5.4	2.3	Iris-virginica
149	5.9	3.0	5.1	1.8	Iris-virginica
150	NaN	4.0	5.0	0.4	Iris-setosa

Iremos, em seguida, definir valores para o preenchimento de *missings* e realizar o preenchimento dos *missings* com esses valores definidos:

cod.py

```
1 # definindo valores para o preenchimento de missings
2 values = {'comprimento_sepala': iris['comprimento_sepala'].median(),
3           'largura_sepala': 5.0,
4           'comprimento_petala': 4.0,
5           'largura_petala': 0.1,
6           'especie': 'Iris-setosa'}
7
8 # fazendo o preenchimento
9 df = df.fillna(value=values)
10 df.tail()
```

	comprimento_sepala	largura_sepala	comprimento_petala	largura_petala	especie
146	6.3	2.5	5.0	1.9	Iris-virginica
147	6.5	3.0	5.2	2.0	Iris-virginica
148	6.2	3.4	5.4	2.3	Iris-virginica
149	5.9	3.0	5.1	1.8	Iris-virginica
150	5.8	4.0	5.0	0.4	Iris-setosa

A seguir, acompanhe um exemplo envolvendo as transformações numéricas.

Transformações numéricas

Podemos realizar as operações de **normalização** e **padronização** usando o módulo de pré-processamento da biblioteca **scikit-learn**.

Aprenda mais!

- Acesse a biblioteca [clikando aqui](#).
- E lembre-se:
- Para normalizar os dados, usamos o *MinMaxScaler*.
 - Para padronizar os dados, usamos o *StandardScale*.

Quando normalizar e quando padronizar?

Fique ligado!

- Se a distribuição é normal, padronize. Caso contrário, normalize.
- Os problemas de modelagem preditiva são muitas vezes complexos, não sendo clara a melhor transformação para realizar. Na dúvida, use a normalização. Se tiver tempo, explore os modelos com os dados sem transformação, com a padronização e com a normalização e veja se os resultados são significativamente diferentes e se o custo x benefício vale a pena.
- Como a padronização resulta em valores positivos e negativos, pode ser interessante normalizar os dados após a padronização.
- É possível definir os valores de mínimo e máximo de acordo com o conhecimento no negócio (e não simplesmente se ater aos valores observados).

Para exemplificar as transformações numéricas, utilizaremos um **dataset** pequeno e artificial, para que seja fácil notar as transformações realizadas:

```
# dados que iremos usar nos exemplos
data = np.asarray([[100, 0.001],
                   [8, 0.05],
                   [50, 0.005],
                   [88, 0.07],
                   [4, 0.1]])

print(data)

[[1.0e+02 1.0e-03]
 [8.0e+00 5.0e-02]
 [5.0e+01 5.0e-03]
 [8.8e+01 7.0e-02]
 [4.0e+00 1.0e-01]]
```

A seguir, iremos normalizar os dados originais e, em seguida, padronizar os dados originais. Os comentários nos blocos de código auxiliam no seu entendimento. Veja o processo de normalização:

```
# Normalização

# definindo o transformador como min max scaler
scaler = MinMaxScaler()

# transformando os dados
scaled = scaler.fit_transform(data)
print(scaled)

[[1.         0.         ]
 [0.04166667 0.49494949]
 [0.47916667 0.04040404]
 [0.875      0.6969697 ]
 [0.         1.         ]]
```

E este é o processo de padronização:

```
# Padronização

# definindo o transformador como standard scaler
scaler = StandardScaler()

# transformando os dados
scaled = scaler.fit_transform(data)
print(scaled)

[[ 1.26398112 -1.16389967]
 [-1.06174414  0.12639634]
 [ 0.         -1.05856939]
 [ 0.96062565  0.65304778]
 [-1.16286263  1.44302493]]
```



Vamos entender agora como proceder com as transformações categóricas.

Transformações categóricas

Podemos realizar as operações de **one-hot encoding** e **dummy encoding** usando o módulo de pré-processamento da biblioteca **scikit-learn**.

Aprenda mais!

Acesse a biblioteca [clikando aqui](#).

E lembre-se:

- Para o *one-hot encoding*, usamos o *OneHotEncoder*.
- Para o *dummy encoding*, também usamos o *OneHotEncoder*, mas com o parâmetro *drop* para indicar qual categoria receberá todos os valores zero.



O *OneHotEncoder* ordena as categorias alfabeticamente antes de aplicar a transformação. É possível especificar a lista de categorias por meio do parâmetro *categories*. Espera-se que o conjunto de treinamento contenha pelo menos um exemplo de cada categoria se as categorias não forem explicitamente definidas.

Se os novos dados (conjunto de teste, por exemplo) tiverem categorias não vistas no treinamento, é possível configurar o parâmetro *handle_unknown* como *ignore* para que não ocorra um erro.

Para exemplificar as transformações categóricas, utilizaremos um *dataset* pequeno e artificial, para que seja fácil notar as transformações realizadas:

cod.py

```
1 # definindo valores para o preenchimento de missings
2 values = {'comprimento_sepala': iris['comprimento_sepala'].median(),
3           'largura_sepala': 5.0,
4           'comprimento_petala': 4.0,
5           'largura_petala': 0.1,
6           'especie': 'Iris-setosa'}
7
8 # fazendo o preenchimento
9 df = df.fillna(value=values)
10 df.tail()
```

	comprimento_sepala	largura_sepala	comprimento_petala	largura_petala	especie
146	6.3	2.5	5.0	1.9	Iris-virginica
147	6.5	3.0	5.2	2.0	Iris-virginica
148	6.2	3.4	5.4	2.3	Iris-virginica
149	5.9	3.0	5.1	1.8	Iris-virginica
150	5.8	4.0	5.0	0.4	Iris-setosa

A seguir, iremos aplicar o *one-hot encoding* nos dados originais e, em seguida, aplicar o *dummy encoding* nos dados originais. Os comentários nos blocos de código auxiliam no seu entendimento.

Acompanhe a seguir:

One-hot encoding

```
# definindo o transformador como one hot encoding
encoder = OneHotEncoder(sparse=False)

# transformando os dados
onehot = encoder.fit_transform(data)
print(onehot)

[[0. 1. 0.]
 [0. 0. 1.]
 [1. 0. 0.]
 [1. 0. 0.]
 [0. 1. 0.]
 [1. 0. 0.]]
```

Dummy encoding

```
# definindo o transformador como one hot encoding (com Dummy variable encoder)
encoder = OneHotEncoder(drop='first', sparse=False)

# transformando os dados
dummy = encoder.fit_transform(data)
print(dummy)

[[1. 0.]
 [0. 1.]
 [0. 0.]
 [0. 0.]
 [1. 0.]
 [0. 0.]]
```

Agora, que tal acompanhar um exemplo prático?

Pré-processamento de dados na prática com o *dataset* diabetes e o *dataset* breast cancer

Neste vídeo, acompanhe a explicação do processo de pré-processamento de dados utilizando como exemplo esses dois *datasets*.



Assista o vídeo.

Gostou do vídeo? Siga em frente para instigar ainda mais suas reflexões sobre o assunto desta aula.

Reflexão

Agora que você conheceu um pouco da etapa de pré-processamento de dados, chegou a hora de exercitar o cérebro e refletir sobre situações do cotidiano de um cientista de dados. Com base nesta última parte da aula, reflita: Quais cuidados devemos ter quando executamos as técnicas de pré-processamento de dados e quais são as consequências da sua não utilização?

Referências

Clique aqui para acessar as referências e créditos desta aula.

